

Chapter 12: Delivering Parcels

Nature Inspired Optimisation for Delivery Problems (2022)

Based on Urquhart, Höhl, Hart (2019, 2021)

Chapter Overview I

This chapter investigates the **Micro-Depot Vehicle Routing Problem (MDVRP)** applied to parcel delivery in urban city centres. The core challenge is real and urgent: as online shopping has grown dramatically, freight vehicles are causing serious traffic congestion and harmful emissions in city centres. The chapter proposes a nature-inspired solution that illuminates the full space of possible delivery strategies rather than finding a single “best” answer.

The chapter is structured around five main topics:

- 1 **Problem Description** — what micro-depots are, why they help, and how the Edinburgh parcel-delivery scenario is formulated.
- 2 **Methodology** — the MAP-Elites illumination algorithm combined with a flexible genetic representation.
- 3 **Implementation** — the software architecture, chromosome encoding, and decoding procedure (with Algorithm 24).
- 4 **Results** — parallel coordinates visualisation, heatmaps, and FlexGP-extracted human-readable policies.
- 5 **Conclusions** — key insights about when and how to use micro-depots effectively.

Key Insight

Rather than asking “What is the single best delivery plan?”, MAP-Elites asks “What is the best delivery plan *for each type of situation?*” This produces an entire library of solutions that a logistics planner can explore.

12.1 The Urban Delivery Problem I

Urban last-mile delivery faces a fundamental conflict: customers want fast, reliable delivery of parcels, but dense cities cannot absorb unlimited freight-vehicle traffic. The resulting congestion harms everyone — it slows all vehicles, raises fuel consumption, and increases emissions of CO₂ and fine particles.

What is a Micro-Depot (MD)?

A **micro-depot** (sometimes called a *city consolidation centre*) is a small, strategically placed storage point within or near a city centre. A large supply vehicle — which need not enter the congested core — restocks each micro-depot once or a few times per day. From the micro-depot, small and low-emission couriers (walkers, cyclists, electric vehicles) make the final deliveries to individual addresses. This two-tier approach keeps large vehicles on arterial roads and puts clean, nimble couriers on narrow city streets.

Three categories of cost or impact are of interest to city planners:

- **Traffic congestion** — how many vehicle trips enter the city core.
- **Emissions** — total CO₂-equivalent produced.
- **Delivery time** — how long customers wait.

12.1 The Urban Delivery Problem II

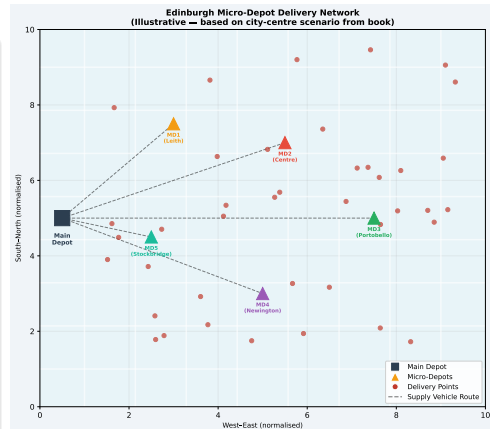
These three objectives often conflict with each other, motivating the multi-characteristic illumination approach.

12.1 The Edinburgh Scenario I

The chapter uses a **real-world data scenario based on Edinburgh, Scotland**. The city centre has a historic layout with narrow streets and restricted access zones, making it a natural testbed for micro-depot strategies.

Problem Data

- A set of delivery addresses (customers) located within the Edinburgh city centre area.
- A main depot located outside the city centre (labelled *D0*).
- Several candidate micro-depot locations distributed across the city, each identified by a unique ID (MD1, MD2, ...).
- A supply vehicle that can visit any micro-depot to drop off parcels.
- Three courier types operating from micro-depots: **walking, cycling, and electric van (EV)**.



12.1 The Edinburgh Scenario II

The problem is defined over **10 separate problem instances**, each representing, for example, one day's worth of deliveries (a subset of the full customer set). Each instance is treated as an independent optimisation run, and results are aggregated to see patterns across instances.

12.1 Courier Types and Their Trade-offs I

The three courier types differ in speed, capacity, and environmental impact. Understanding these differences is essential for interpreting the results.

Courier Type Comparison

Type	Speed	Capacity	Emissions	Network access
Walking	Very slow	Very low	Zero	All paths
Cycling	Slow	Low	Zero	Cycle paths + roads
Electric Van (EV)	Moderate	High	Very low	Roads only
Supply Vehicle	Fast	Very high	Moderate–High	Arterial roads

Paths available to couriers depend on vehicle type. Walking and cycling couriers can use pedestrian paths and cycle lanes that are not accessible to electric vans or the supply vehicle. The routing algorithm (nearest-neighbour heuristic) must respect these path restrictions when computing distances.

12.1 Courier Types and Their Trade-offs II

Key Insight

Electric vans are fast enough to cover more deliveries per time unit and carry more parcels, but they are restricted to road networks. Cyclists and walkers are slower but can reach destinations that road vehicles cannot, and they produce zero emissions. The optimal policy depends on the ratio of narrow-street deliveries to road-accessible deliveries.

12.1 Solution Characteristics: Measuring the Archive I

Rather than tracking a single objective value, MAP-Elites characterises each solution by **seven behavioural dimensions**. These dimensions describe *how* a solution achieves its deliveries, not merely whether it is feasible.

The Seven Solution Characteristics (Table 12.1)

Label	Description	Scale
X_1	Number of micro-depots used	1–10
X_2	Percentage of deliveries routed via micro-depots	1–10
X_3	Number of cycling couriers used	1–10
X_4	Number of walking couriers used	1–10
X_5	Number of EV couriers used	1–10
X_6	Total delivery time (normalised)	1–10
X_7	Total emissions (normalised)	1–10

12.1 Solution Characteristics: Measuring the Archive II

Each characteristic is **normalised to the integer range 1–10**, which means the archive has at most $10^7 = 10,000,000$ cells. In practice, only a fraction of cells are occupied, because many combinations of characteristics are infeasible or never discovered.

Label Mapping (Table 12.2)

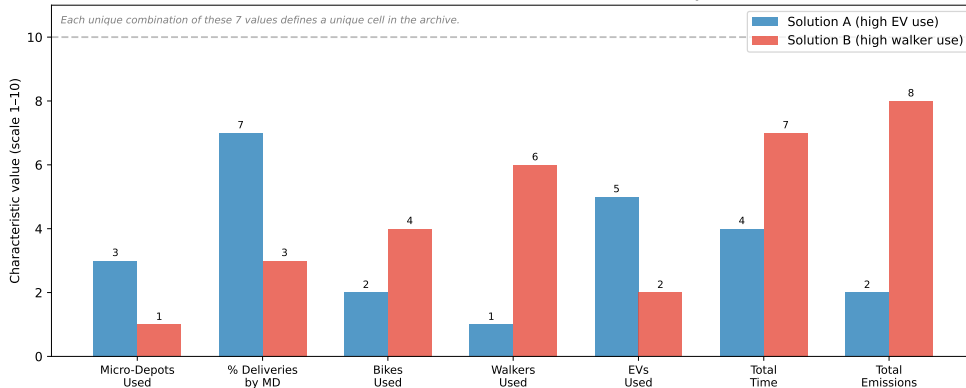
To make policies human-readable, the integer scale is translated into natural-language labels:

Range	Label
1–2	Very Low
3–4	Low
5–6	Medium
7–8	High
9–10	Very High

For example, a solution with $X_5 = 9$ would be labelled “EVs Used = Very High”. This label system is used by FlexGP (Section 12.2) to produce human-readable policy statements.

12.1 Solution Characteristics: Measuring the Archive III

Seven Solution Characteristics used as MAP-Elites Archive Dimensions
(Each characteristic normalised to scale 1-10; archive size up to 10^7 cells)



Two contrasting solutions across the seven characteristic dimensions. Solution A maximises EV use and achieves low emissions; Solution B relies on walking couriers and suffers higher time and emissions.

12.2 Overview of the Hybrid Approach I

The methodology combines three well-established techniques into a coherent pipeline:

- 1 **MAP-Elites** (from Chapter 5 / Chapter 6 of the book) — an illumination algorithm that fills a multidimensional behavioural archive with high-quality solutions. Each cell of the archive stores the best solution found so far for that particular combination of characteristics.
- 2 **Evolutionary operators** — mutation and crossover on a chromosome-based representation generate new candidate solutions for MAP-Elites to evaluate.
- 3 **FlexGP** — a machine-learning rule-extraction tool that reads the populated archive and outputs Boolean decision rules (policies) that explain which regions of the solution space yield good outcomes.

Why this pipeline?

A standard genetic algorithm would return a single solution. MAP-Elites returns a *library* of solutions, each optimised for a different operating scenario. FlexGP then converts this library into plain-English policies that a manager — not just an algorithm expert — can act on.

12.2 MAP-Elites: Illumination Algorithm I

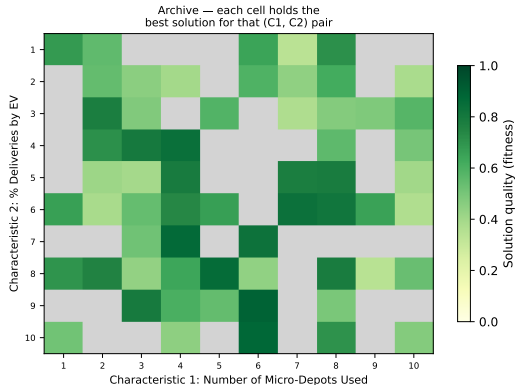
MAP-Elites was introduced by Mouret and Clune (2015) as a way to *illuminate the solution space* rather than converge to a single point. The name stands for **Map of Elites**: it builds a map (the archive grid) in which each cell holds the best (*elite*) solution found for that cell's combination of characteristics.

MAP-Elites Algorithm (High Level)

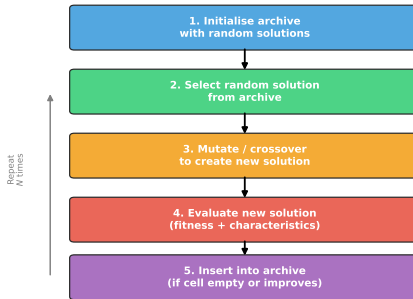
- ➊ **Initialise**: Generate a random population of solutions. For each, evaluate its fitness and compute its d -dimensional characteristic vector (here $d = 7$). Insert each solution into the archive if its cell is empty or if it improves on the existing occupant.
- ➋ **Repeat** for N iterations:
 - ➊ Select a random solution $\mathbf{x}$ from the archive.
 - ➋ Create a new candidate $\mathbf{x}'$ by mutating (and optionally crossing over) $\mathbf{x}$.
 - ➌ Evaluate $\mathbf{x}'$: compute its fitness and characteristic vector.
 - ➍ Attempt to insert $\mathbf{x}'$ into the archive: if its cell is empty, or if it has higher fitness than the current occupant, replace the occupant.
- ➌ **Output**: the entire archive — a map of the best solution found for each region of behaviour-space.

12.2 MAP-Elites: Illumination Algorithm II

MAP-Elites: Illuminating the Solution Space



MAP-Elites Algorithm Flow



Left: example 2D archive slice showing solution quality (greener = better) across two characteristics. Right: the five-step MAP-Elites algorithm loop.

12.2 Use of Machine Learning: FlexGP I

After MAP-Elites produces an archive of solutions, the chapter uses **FlexGP** (Flexible Genetic Programming) to extract human-interpretable policies from the archive data.

FlexGP is a machine-learning classifier that learns Boolean expression trees. The archive is converted into a supervised learning dataset: each solution in the archive becomes one training example. The input features are the seven solution characteristics $(X_1, \dots, X_7)$, and the output label is a binary classification (0 or 1) derived from the solution's performance on the objective of interest (e.g. emissions).

12.2 Use of Machine Learning: FlexGP II

Training Data Format

Each row of the training data looks like:

$$(X_1, X_2, X_3, X_4, X_5, X_6, X_7, \text{label})$$

where the label is 1 if the solution has “very low emissions” and 0 if it has higher emissions. An example data fragment from the book:

```
4,2,10,6,8,7,1
6,5,5,6,10,2,0
7,1,10,3,2,1,1
...
```

Here column 7 (the last) is the label: 1 = very low emissions, 0 = higher.

FlexGP's task is to find a compact Boolean rule (a short decision tree expressed as a logical formula) that correctly classifies as many training examples as possible. The result is a policy that a logistics manager can read, understand, and apply to new situations without needing to re-run the algorithm.

12.2 FlexGP Policy: Structure and Interpretation I

FlexGP evolves Boolean condition trees using the seven characteristic variables. Each interior node is AND or OR; each leaf is a range condition on one variable. The tree is then translated into natural language using the label table (Table 12.2).

Example FlexGP Policy (Low Emissions)

The raw policy extracted from the archive is:

$$((1.0 \leq X_5 \leq 8.6 \text{ and } 1.0 \leq X_4 \leq 2.0) \text{ and } (1.0 \leq X_2 \leq 8.4 \text{ or } 1.0 \leq X_3 \leq 2.7))$$

where:

- X_2 = percentage of deliveries via micro-depots,
- X_3 = number of bicycles (bikes) used,
- X_4 = number of walking couriers used,
- X_5 = number of electric vehicles (EVs) used.

12.2 FlexGP Policy: Structure and Interpretation II

Substituting the label names, the policy becomes:

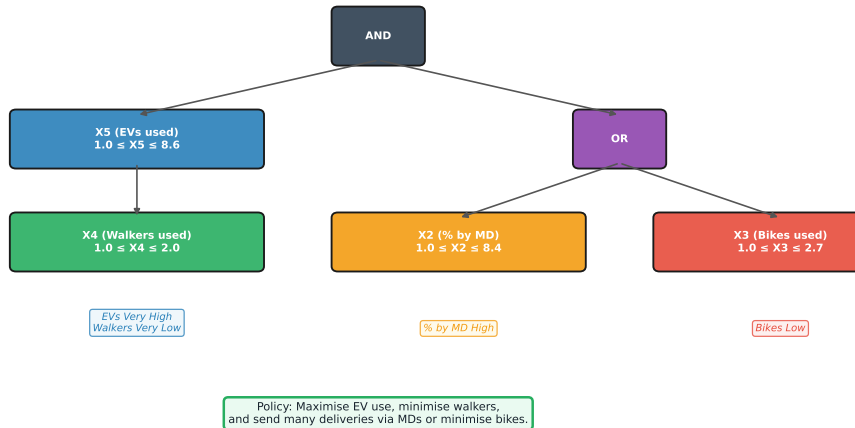
$((EVsUsed \leq \text{VeryHigh} \textbf{ and } WalkersUsed \leq \text{VeryLow}) \textbf{ and } (\%ByMD \leq \text{High} \textbf{ or } BikesUsed \leq \text{Low}))$

Plain-English Interpretation

"Maximise the use of EVs, make minimal use of walking couriers, and either send a large fraction of deliveries through micro-depots or minimise the use of bicycles." This makes intuitive sense: EVs are the cleanest powered couriers, walkers are slow and cover few deliveries per hour, and MDs allow more goods to be dispatched concurrently from distributed points.

12.2 FlexGP Policy: Structure and Interpretation III

FlexGP: Machine-Learning Policy Extraction from MAP-Elites Archive
Boolean expression tree: low-emission label predicted from solution characteristics



12.2 FlexGP Policy: Structure and Interpretation IV

The FlexGP Boolean decision tree for the low-emissions policy. Each leaf shows the translated natural-language label.

12.2 Why MAP-Elites Suits This Problem I

Several properties of the micro-depot delivery problem make MAP-Elites a particularly good fit:

- ① **Multiple objectives, multiple stakeholders.** A city authority cares about emissions; a courier company cares about delivery time; a resident cares about convenience. MAP-Elites maintains solutions optimised under all of these preferences simultaneously, rather than requiring the designer to specify weights upfront.
- ② **Heterogeneous operating conditions.** Some days have few deliveries; others are busy. Some neighbourhoods have cycle paths; others do not. MAP-Elites captures this variation because each archive cell corresponds to a different operating regime.
- ③ **Decision support, not automation.** Logistics managers want to understand *why* a policy works, not just execute a black-box recommendation. The FlexGP post-processing step converts the archive into transparent rules.
- ④ **Computational scalability.** The archive representation allows MAP-Elites to scale to large problem instances without losing diversity, because solutions compete only within their own cell.

12.3 Software Architecture I

The implementation is built around a set of Python/Java classes. The structure is shown in Figure 12.3 of the book. The key design principle is **separation of concerns**: the MAP-Elites search logic is kept separate from the logistics simulation, which is itself kept separate from the results analysis.

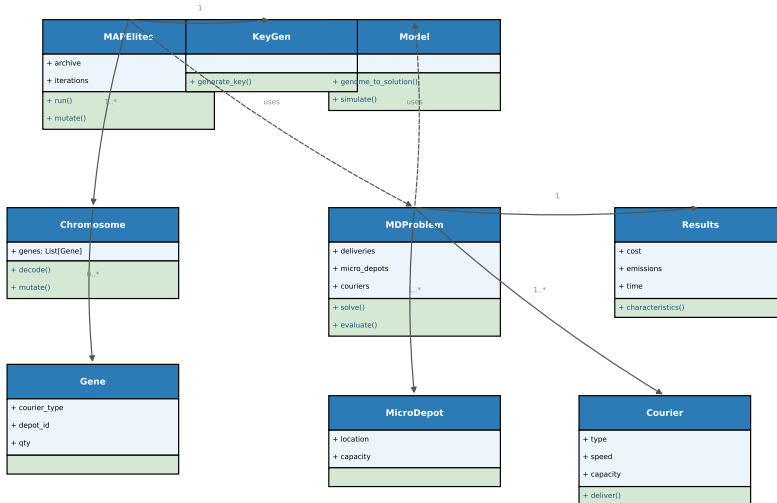
12.3 Software Architecture II

Core Classes

Class	Responsibility
MDProblem	Encapsulates deliveries, MDs, couriers; provides <code>solve()</code> and <code>evaluate()</code>
Model	Converts a chromosome into a delivery plan (simulates the logistics network)
Results	Stores and exposes the solution: cost, emissions, time, characteristics
MAPElites	Runs the archive-based evolutionary search
KeyGen	Computes the 7-dimensional archive key from a <code>Results</code> object
Chromosome	Holds a variable-length list of <code>Gene</code> objects
Gene	One courier assignment: (courier type, micro-depot ID, delivery quantity)
MicroDepot	Location and capacity of one micro-depot
Courier	Type, speed, capacity of one courier

12.3 Software Architecture III

Software Architecture: Key Classes in the MDVRP Solver
(Solid arrows = association; dashed = dependency/uses)



12.3 Representation: Chromosomes and Genes I

The evolutionary algorithm works with **chromosomes**, which are the data structures it evolves. Each chromosome represents one complete delivery plan.

Chromosome Structure

A chromosome is a variable-length ordered list of **genes**. Each gene encodes one courier assignment and contains exactly three pieces of information:

- 1 **Courier type**: one of {walking, cycling, electric van}.
- 2 **Micro-depot ID**: which micro-depot this courier operates from.
- 3 **Delivery quantity**: how many deliveries are transferred to this courier (an integer from 1 upward).

12.3 Representation: Chromosomes and Genes II

Example Gene

Courier type	Micro-depot	Quantity
walking	MD2	3

This gene says: “create a walking courier based at micro-depot 2, and assign it 3 deliveries from the default supply route.”

12.3 Representation: Chromosomes and Genes III

Example Chromosome (4 genes)

Gene	Courier type	Micro-depot	Quantity
1	walking	MD2	3
2	cycling	MD1	4
3	electric van	MD3	4
4	walking	MD1	3

This chromosome describes a plan that uses four couriers across three micro-depots (MD1, MD2, MD3) to handle a total of $3 + 4 + 4 + 3 = 14$ deliveries.

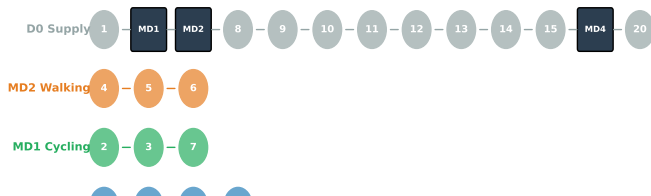
12.3 Representation: Chromosomes and Genes IV

Chromosome Representation and Solution Decoding

A chromosome encodes a list of genes; each gene = one courier assignment

Gene	Courier Type	Micro-Depot ID	Quantity
1	Walking	MD2	3
2	Cycling	MD1	4
3	Electric Van (EV)	MD3	4
4	Walking	MD1	3

Decoding: 20 deliveries transferred from default supply tour to courier sub-tours



12.3 Default Tour and the Nearest Neighbour Heuristic I

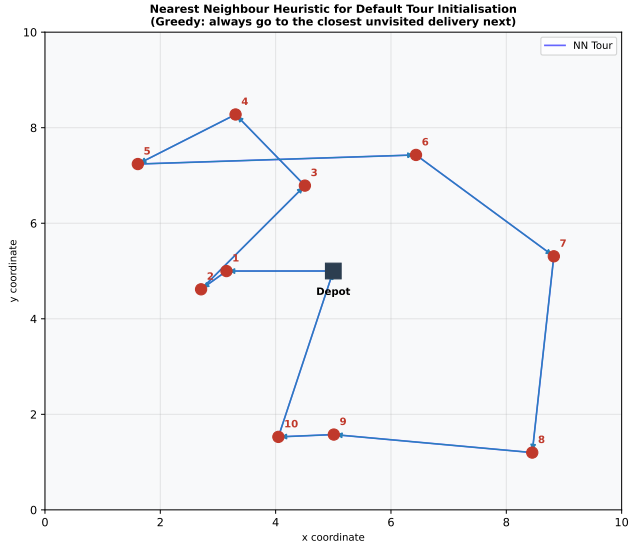
Every chromosome is decoded *relative to a **default tour***. Understanding the default tour is essential for understanding how the chromosome decoding works.

Default Tour

The default tour is the delivery plan that uses *no* micro-depots and *no* couriers: a single supply vehicle departs from the main depot D0 and visits all delivery addresses directly, in the order determined by the nearest-neighbour (NN) heuristic.

The NN heuristic is a simple greedy construction method: starting from the depot, always travel next to the closest unvisited delivery point; repeat until all deliveries are served; then return to the depot. Although not optimal, NN is fast and produces a reasonable initial tour.

12.3 Default Tour and the Nearest Neighbour Heuristic II



The nearest-neighbour heuristic builds the default tour greedily. Numbers show the visit order; the supply vehicle starts and ends at the depot.

12.3 Decoding a Chromosome — Step by Step I

Decoding a chromosome means converting it from a list of genes into a concrete delivery schedule with specific routes. The decoding procedure follows Algorithm 24 (shown below) for each gene in sequence.

12.3 Decoding a Chromosome — Step by Step II

Algorithm 24: Create Courier Tour

Procedure createCourierTour(supplyTour, c, md, len)

Input supplyTour: the current supply vehicle tour
 c: courier type (walking/cycling/EV)
 md: the micro-depot this courier starts from
 len: the number of deliveries to transfer

Output A new courier sub-tour; supplyTour updated

- ① Create an empty tour for courier c, starting at micro-depot md.
- ② Set current $\leftarrow$ md. Set count $\leftarrow$ 0.
- ③ **While** count < len:
 - ① Find del = the delivery in supplyTour closest to current (using distances appropriate for vehicle type c).
 - ② If the courier tour can still accommodate del (capacity not exceeded): add del to courier tour; remove from supplyTour. Increment count.
 - ③ Advance current $\leftarrow$ location of del.
- ④ Find the insertion point in supplyTour closest to md; insert a visit to md at that position (so the supply vehicle will deliver a batch to MD).

12.3 Worked Decoding Example — 20 Deliveries I

Let us trace through decoding the example chromosome from page 244–245 of the book. We have 20 deliveries, labelled 1–20.

Step 1: Initial default tour (before decoding any genes). All 20 deliveries are assigned to the supply vehicle at D0.

Depot	Vehicle/Courier	Deliveries
D0	Supply	1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20

Step 2: Decode gene 1 (walking, MD2, qty 3). The algorithm finds the 3 deliveries closest to MD2 in the supply tour (starting from insertion position 4 in the NN order). Customers 4, 5, 6 are extracted from the supply tour. The supply tour now includes a stop at MD2 to drop off these parcels.

Depot	Vehicle/Courier	Deliveries
D0	Supply	1 2 3 MD2 7 8 9 10 11 12 13 14 15 16 17 18 19 20
MD2	Walking	4 5 6

12.3 Worked Decoding Example — 20 Deliveries II

Step 3: Decode gene 2 (cycling, MD1, qty 4). Customers 2, 3, 7 are nearest to MD1, so they are extracted. Note that customer 2 position in the supply tour now points to MD1 (the supply vehicle visits MD1 to drop off 2, 3, 7).

Depot	Vehicle/Courier	Deliveries
D0	Supply	1 MD1 MD2 8 9 10 11 12 13 14 15 16 17 18 19 20
MD2	Walking	4 5 6
MD1	Cycling	2 3 7

Step 4: Decode genes 3 and 4 (EV from MD3 and walking from MD1). Two more courier routes are created. Note that MD1 already appears in the supply tour from Step 3, so no second visit to MD1 is needed for the walking courier; the supply vehicle delivers to MD1 in one combined stop.

Depot	Vehicle/Courier	Deliveries
D0	Supply	1 MD1 MD2 8 9 11 14 15 MD4 20
MD2	Walking	4 5 6
MD1	Cycling	2 3 7
MD4	EV	16 17 18 19
MD1	Walking	10 12 13

12.3 Worked Decoding Example — 20 Deliveries III

Key Insight

The supply vehicle now visits only 4 micro-depot stops (MD1 once, MD2, MD4, and delivers remaining directly), eliminating 14 of the original 20 in-city stops. The four couriers handle 14 deliveries using zero-emission or low-emission vehicles.

12.3 Evolutionary Operators I

MAP-Elites needs operators to *generate* new candidate chromosomes from existing archive entries. The chapter uses standard evolutionary operators adapted for variable-length chromosome lists.

Initialisation

During initialisation, each new chromosome is created with a randomly chosen number of genes between 0 and 10. Each gene's values (courier type, depot ID, quantity) are set randomly. A chromosome with 0 genes represents the default tour (no couriers at all), which is always a valid, feasible solution.

12.3 Evolutionary Operators II

Mutation Operations (one chosen at random per generation)

- 1 **Add gene:** append a new gene with random values. This grows the chromosome, testing new courier assignments.
- 2 **Move gene:** randomly select one gene and move it to a different position in the list. The decoding order can affect which deliveries are transferred first, so position matters.
- 3 **Alter gene:** randomly select one gene and randomly change one of its three values (type, depot, or quantity). This fine-tunes a specific courier assignment.
- 4 **Delete gene:** randomly remove one gene. This reduces the number of couriers, testing whether a simpler plan performs as well.

Crossover

A new chromosome is created by selecting 10 genes from randomly chosen positions in a randomly selected parent chromosome. This produces offspring of length 10 that combine genes from different regions of the parent.

12.3 Evolutionary Operators III

Key Insight

Variable-length chromosomes allow the algorithm to automatically discover both the *number* of couriers needed and their assignments. The designer does not need to specify how many couriers to use in advance — the algorithm learns this from experience.

12.3 Python: MDPProblem and Results Classes

The following Python code sketches the core data structures used in the implementation. The MDPProblem class holds all input data and exposes a solve() method; the Results class captures the output.

Listing: Core data structures (illustrative)

```
1  from dataclasses import dataclass, field
2  from typing import List
3  from enum import Enum
4
5  class CourierType(Enum):
6      WALKING = "walking"
7      CYCLING = "cycling"
8      EV      = "electric_van"
9
10 @dataclass
11 class Gene:
12     """One courier assignment: type, depot, quantity."""
13     courier_type: CourierType
14     depot_id: int      # which micro-depot (1-indexed)
15     quantity: int      # number of deliveries to take
16
17 @dataclass
18 class Chromosome:
19     """Variable-length list of genes (one gene = one courier)."""
20     genes: List[Gene] = field(default_factory=list)
21
22     def mutate_add(self, depot_ids, max_qty=10):
```

12.3 Python: Nearest Neighbour Tour Construction

The default tour is built by the nearest-neighbour heuristic. The function below constructs this tour and returns the visit order. This is the starting point for all chromosome decoding.

Listing: Nearest neighbour heuristic for default tour

```
1 import numpy as np
2
3 def nearest_neighbour_tour(depot, deliveries):
4     """
5     Build a nearest-neighbour tour starting and ending at the depot.
6
7     Parameters
8     -----
9     depot      : (float, float) -- (x, y) coordinates of the main depot.
10    deliveries  : list of (float, float) -- customer locations.
11
12    Returns
13    -----
14    tour_order  : list of int -- indices into deliveries in visit order.
15    """
16    unvisited = list(range(len(deliveries)))
17    tour_order = []
18    current = np.array(depot)
19
20    while unvisited:
21        # Compute Euclidean distance from current location to all unvisited
22        dists = [np.linalg.norm(np.array(deliveries[i]) - current)
```

12.3 Python: Chromosome Decoding Loop

The decoding loop iterates over the genes in a chromosome and calls `create_courier_tour()` for each gene. This progressively transfers deliveries from the supply tour to courier sub-tours.

Listing: Chromosome decoding — transfer deliveries to couriers

```
1  def decode_chromosome(chromosome, supply_tour, depots, couriers):
2      """
3      Decode a chromosome into a set of courier routes.
4
5      Parameters
6      -----
7      chromosome : Chromosome object (list of genes)
8      supply_tour : list of delivery indices (mutable -- will be modified)
9      depots      : dict {depot_id: (x, y)} location of each micro-depot
10     couriers     : list to accumulate (courier_type, depot_id, [deliveries])
11
12     Returns
13     -----
14     couriers : list of (courier_type, depot_id, delivery_list)
15     supply_tour : remaining deliveries for supply vehicle
16     """
17     for gene in chromosome.genes:
18         ctype = gene.courier_type
19         md_id  = gene.depot_id
20         qty    = gene.quantity
21         md_loc = depots.get(md_id)
```

12.3 Python: MAP-Elites Archive and Main Loop

The MAP-Elites archive is a dictionary mapping a 7-tuple of integers (the characteristic key) to the best Results object found for that key.

Listing: MAP-Elites archive and search loop

```
1 import random
2
3 def map_elites_run(problem, n_iterations=10000, init_pop=500):
4     """
5     Run MAP-Elites to fill the 7-D archive.
6
7     Parameters
8     -----
9     problem          : MDPProblem instance
10    n_iterations      : int -- number of MAP-Elites iterations
11    init_pop          : int -- initial random population size
12
13    Returns
14    -----
15    archive : dict  {(c1,...,c7): Results}
16    """
17    archive = {}    # key = 7-tuple, value = (fitness, chromosome, results)
18
19    def insert(chrom, results):
20        key = results.characteristics()
21        fitness = -results.total_emissions    # maximise = minimise emissions
22        if key not in archive or fitness > archive[key][0]:
```

12.3 Python: FlexGP Training Data Preparation

After MAP-Elites produces the archive, we prepare training data for FlexGP by labelling each solution based on its emissions level.

Listing: Prepare FlexGP training data from MAP-Elites archive

```
1 import csv
2
3 def prepare_flexgp_data(archive, emissions_threshold=3,
4                         output_file="flexgp_data.csv"):
5     """
6     Convert the MAP-Elites archive into FlexGP training data.
7
8     Label = 1 (very low emissions) if emissions_norm <= threshold,
9     Label = 0 (higher emissions) otherwise.
10
11     The 7 characteristics are: n_mds, pct_by_md, bikes, walkers,
12     evs, time_norm, emissions_norm.
13     The last column is the binary label.
14     """
15     rows = []
16     for key, (fitness, chrom, results) in archive.items():
17         c1, c2, c3, c4, c5, c6, c7 = results.characteristics()
18         # Label: 1 = very low emissions (c7 <= threshold), 0 = otherwise
19         label = 1 if c7 <= emissions_threshold else 0
20         rows.append([c1, c2, c3, c4, c5, c6, label])
21
22     with open(output_file, 'w', newline='') as f:
```

12.4 Visualising the Archive: Parallel Coordinates I

After running MAP-Elites across the 10 problem instances, the archives are **aggregated** to see patterns that hold across different days and delivery loads. The primary visualisation tool is the **parallel coordinates plot**.

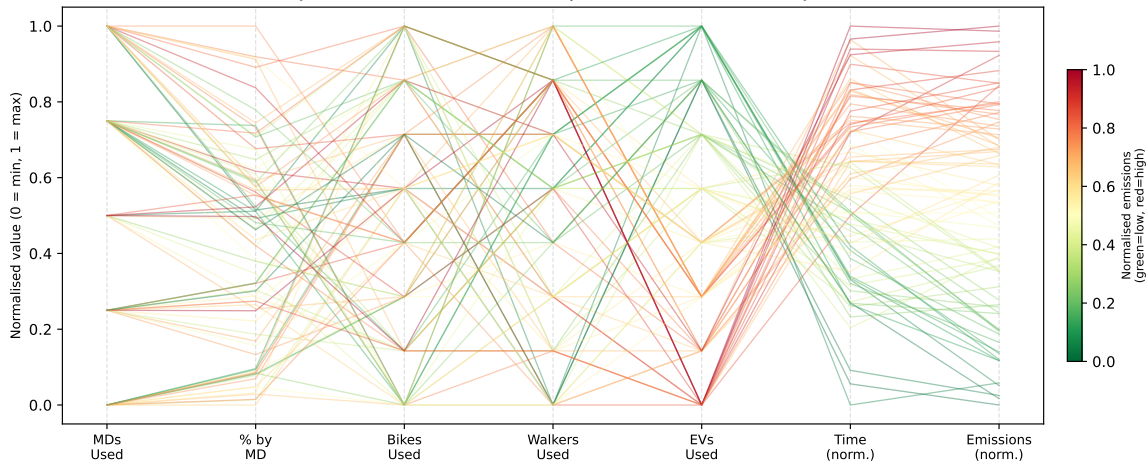
What is a Parallel Coordinates Plot?

A parallel coordinates plot represents a high-dimensional solution as a single polyline drawn through a set of vertical axes. Each axis corresponds to one characteristic (X_1 through X_7). The position of the line on each axis shows the value of that characteristic for this particular solution.

When many solutions are plotted together, the bundle of lines reveals structure: parallel bundles indicate correlation; crossing bundles indicate negative correlation (trade-offs).

12.4 Visualising the Archive: Parallel Coordinates II

Parallel Coordinates Plot of MAP-Elites Archive
(Each line = one elite solution; colour = emissions level)



12.4 Visualising the Archive: Parallel Coordinates III

Parallel coordinates of the MAP-Elites archive. Each line is one elite solution. Colour encodes total emissions level (green = low, red = high). Notice that low-emission lines (green) tend to have high values on the “EVs Used” axis and low values on “Total Emissions”.

How to Read This Plot

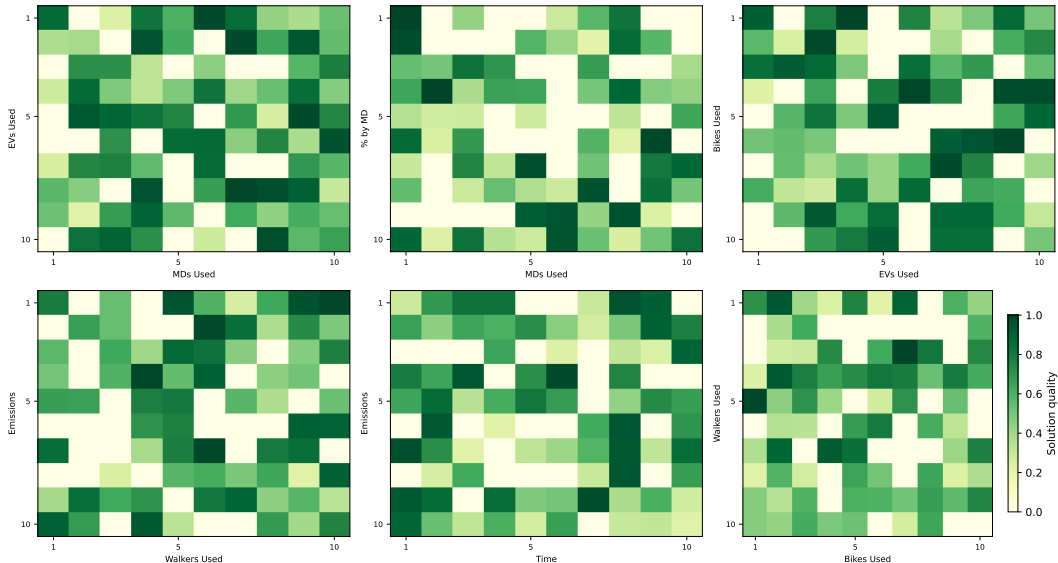
Look for lines that cross between axes — these indicate a trade-off (when one characteristic is high, the other tends to be low). For example, as “EVs Used” increases, the “Emissions” axis tends to decrease: the lines cross, showing a negative correlation. This is expected — electric vehicles produce fewer emissions per delivery.

12.4 Heatmaps of Archive Coverage I

In addition to the parallel coordinates plot, the ELVis tool (Urquhart, Neil, 2019) generates **heatmaps** for each pair of solution characteristics. A heatmap shows how densely populated each region of the two-dimensional projection of the archive is, and how good the solutions found there are.

12.4 Heatmaps of Archive Coverage II

Heatmaps of Archive Coverage: Each Pair of Solution Characteristics
(Bright green = many good solutions found; dark = sparse or empty region)



12.4 Which Factors Lead to Reduced Delivery Time? I

The first policy question the chapter investigates is: **which factors in a solution lead to shorter delivery times?**

To answer this, the archive is filtered to identify solutions with low values on the “Total Time” characteristic (X_6). FlexGP is run on this filtered dataset. The resulting raw policy (based on the knee of the Pareto front for time) is:

FlexGP Policy: Fast Delivery

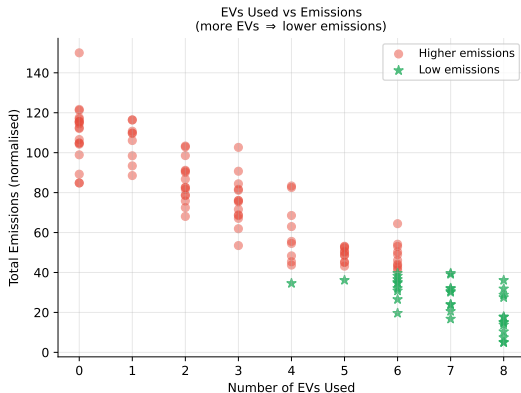
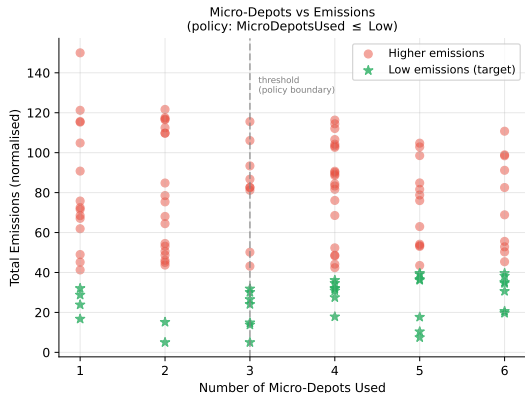
$$E1 \leq X1 \leq E2 \quad \text{and} \quad 3 \leq X5 \leq 10.16$$

where $X1$ = number of micro-depots used and $X5$ = EVs used. After substituting labels: $E1, E2$ refer to specific range boundaries. An example interpretation from the book (p. 248):

$$1.0 \leq X1 \leq 3 \quad \Rightarrow \quad \text{MicroDepotsUsed} \leq \text{Low}$$

12.4 Which Factors Lead to Reduced Delivery Time? II

Effect of Micro-Depots and EVs on Emissions



Left: number of micro-depots vs. emissions (stars = low-emission solutions; policy boundary shown dashed). Right: EVs used vs. emissions (more EVs strongly reduces emissions).

12.4 Which Factors Lead to Reduced Delivery Time? III

Key Insight from the Time Analysis

The analysis reveals that having *few* micro-depots and using EVs (fast couriers) leads to faster overall delivery. This is counterintuitive at first glance — one might expect more micro-depots to speed things up. But the overhead of visiting and stocking many micro-depots slows the supply vehicle down, negating the benefit of faster last-mile couriers. The sweet spot is to use a small number (1–3) of well-placed micro-depots, each stocked efficiently by the supply vehicle.

12.4 Which Factors Lead to Low Emissions? I

The second policy question is: **which combinations of solution characteristics lead to low total emissions?**

FlexGP Raw Policy (Low Emissions)

$$((1.0 \leq X_5 \leq 8.6 \text{ and } 1.0 \leq X_4 \leq 2.0) \text{ and } (1.0 \leq X_2 \leq 8.4 \text{ or } 1.0 \leq X_3 \leq 2.7))$$

After substituting labels:

$$((\text{EVsUsed} \leq \text{VeryHigh} \text{ and } \text{WalkersUsed} \leq \text{VeryLow}) \text{ and } (\% \text{ByMD} \leq \text{High} \text{ or } \text{BikesUsed} \leq \text{Low}))$$

The plain-English interpretation of this policy is:

Maximise the use of EVs and make minimum use of walking couriers. And send as many deliveries via micro-depots as possible, or minimise the use of bicycles.

This policy makes sense for the following reasons:

12.4 Which Factors Lead to Low Emissions? II

- Walking couriers complete very few deliveries per hour compared to EVs. Using many walkers means the supply vehicle must make many small deliveries to many micro-depots, which itself adds emissions.
- EVs are the fastest and highest-capacity green couriers. Fewer total trips are needed when EVs are used.
- Maximising the fraction of deliveries through MDs (%ByMD is High) ensures that the supply vehicle makes efficient batch transfers rather than many small city trips.

12.4 Can We Achieve Low Emissions AND Fast Delivery Together? I

A natural follow-up question is whether low emissions and fast delivery can be achieved *simultaneously*, or whether they are always in conflict.

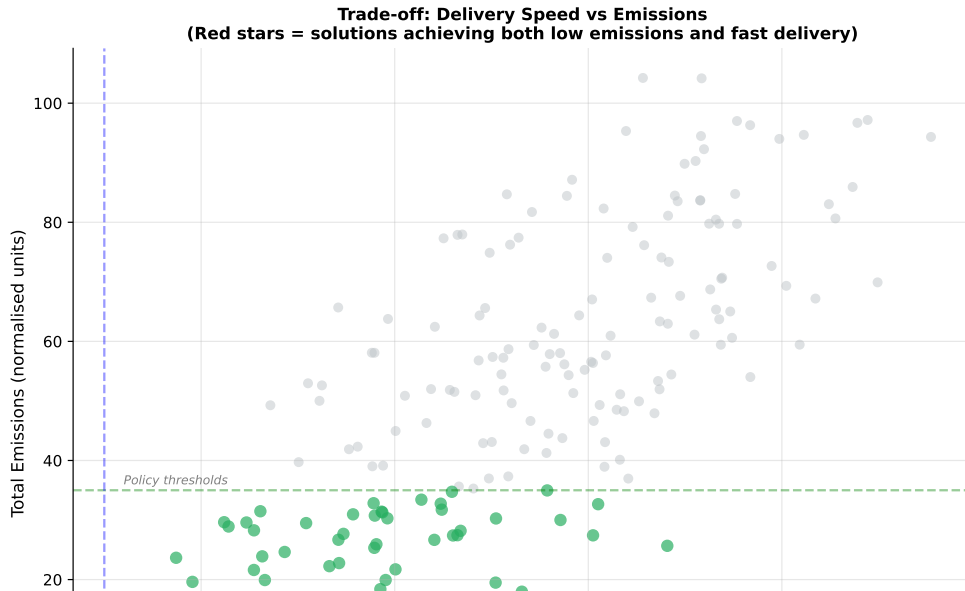
FlexGP Analysis: Combined Objective

Solutions that achieve both low emissions and fast delivery time were highlighted. Running FlexGP on this combined subset yields the policy:

$$1.0 \leq X_1 \leq 3 \quad \Rightarrow \quad \text{MicroDepotsUsed} \leq \text{Low}$$

That is, the distinguishing factor is simply the number of micro-depots: using **at most 3** micro-depots simultaneously achieves both low emissions and fast delivery.

12.4 Can We Achieve Low Emissions AND Fast Delivery Together? II



12.4 Summary of Policy Findings I

The MAP-Elites + FlexGP pipeline produces the following actionable insights for Edinburgh parcel delivery:

Policy Summary Table

Objective	Key Factor(s)	Direction
Reduce emissions	EVs Used	Maximise
Reduce emissions	Walkers Used	Minimise
Reduce emissions	% via Micro-Depots	Maximise
Reduce emissions	Bikes Used	Minimise
Reduce delivery time	Micro-Depots Used	Use few (1–3)
Reduce delivery time	EVs Used	Maximise
Both (combined)	Micro-Depots Used	≤ 3

12.4 Summary of Policy Findings II

These policies are **stable across 10 problem instances**, which provides confidence that they reflect genuine structural properties of the Edinburgh city centre scenario rather than artefacts of a single day's data.

Operational Recommendation

A city logistics operator should: (1) deploy 1–3 micro-depots at strategic city-centre locations, (2) prioritise electric vans for courier assignments, and (3) use walking couriers only as a last resort (e.g. for addresses accessible only on foot). This strategy reduces both emissions and delivery time simultaneously.

12.5 Conclusions I

This chapter has demonstrated a complete pipeline for solving and understanding the urban parcel delivery problem using nature-inspired optimisation.

Main Contributions

- 1 **MAP-Elites as a decision-support tool:** rather than returning a single delivery plan, MAP-Elites returns an entire library — one best plan for each combination of characteristics. Logistics planners can select the plan that best matches today's priorities (low cost, low emissions, fast delivery).
- 2 **Variable-length chromosome representation:** the gene-based encoding allows the algorithm to automatically discover both the number of couriers and their assignments, without requiring the human to specify this in advance. Genes are added, removed, or altered by mutation operators.
- 3 **FlexGP policy extraction:** machine learning converts the archive data into compact, human-readable Boolean rules. This closes the loop between optimisation and managerial decision-making — a manager can act on the policy without understanding the underlying algorithm.
- 4 **Stable policies across instances:** the finding that 1–3 micro-depots with high EV usage dominates both the emissions and time objectives is consistent across all 10 problem instances.

12.5 Conclusions II

Limitations and Open Questions

- The analysis uses relatively small problem instances (one day's deliveries in a city centre). Scaling to larger instances (hundreds of deliveries, many city districts) is an open challenge.
- The courier speed and capacity values are stylised. Real data (e.g. from GPS traces) would improve accuracy.
- FlexGP produces one policy per objective. A multi-objective FlexGP that produces Pareto-optimal policies simultaneously would be valuable.
- Dynamic re-planning (as new orders arrive through the day) is not addressed.

Closing Thought

The most important lesson from Chapter 12 is that the question “What is the best delivery plan?” is the wrong question for complex urban logistics. The right question is “What is the best plan *given today's priorities*, and what does the space of trade-offs look like?” MAP-Elites, combined with transparent policy extraction, answers the right question.

References I

- Mouret, J.-B. and Clune, J. (2015). Illuminating search spaces by mapping elites. *arXiv:1504.04909*.
- Urquhart, N., Höhl, S. and Hart, E. (2019). An illumination algorithm approach to solving the micro-depot routing problem. *Proceedings of GECCO 2019*, ACM, pp. 1347–1355.
- Urquhart, N., Höhl, S. and Hart, E. (2021). Automated, Explainable Rule Extraction from MAP-Elites Archives. *EvoApplications 2021*, LNCS Vol. 12694, pp. 258–272.
- Group, A. (n.d.). Rule Tree Learner. <http://flexgp.github.io/gp-learners/ruletree.html>
- Urquhart, Neil (2019). ELite VISualisation (ElVis). <https://commute.napier.ac.uk/upload>
- Browne, M., Allen, J., and Leonardi, J. (2011). Evaluating the use of an urban consolidation centre and electric vehicles in central London. *IATSS Research* 35(1):1–6.
- Fischer, M. (2017). *City Logistics: Determination of Criteria to Evaluate Locations for Micro-Depots in Frankfurt am Main*. Master's thesis, Frankfurt University of Applied Science.

Chapter 12: Delivering Parcels

Key theme: MAP-Elites illuminates the solution space for urban parcel delivery via micro-depots and green couriers. FlexGP extracts actionable, human-readable policies.

Core finding: Use 1–3 micro-depots with electric vehicles to simultaneously minimise emissions and delivery time in the Edinburgh city-centre scenario.